3 Data Visualization with matplotlib and seaborn

Matplotlib and **seaborn** are two popular Python libraries for data visualization. Matplotlib provides low-level functionality for creating various types of plots, such as line charts, bar charts, histograms, scatter plots, etc. Seaborn builds on top of matplotlib and offers high-level functions for creating more complex and aesthetically pleasing visualizations, such as heatmaps, boxplots, violin plots, etc. Seaborn also integrates well with pandas, a library for data analysis and manipulation. Together, matplotlib and seaborn can help data scientists and analysts explore and communicate insights from their data.

3.1 Installing and Importing Pandas

The following command is used to install pandas to your python system. In your pc, pandas is already installed, so there is no need to run this command.

```
#!pip install matplotlib
#!pip install seaborn
```

Importing the libraries we will use throughout this notebook.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

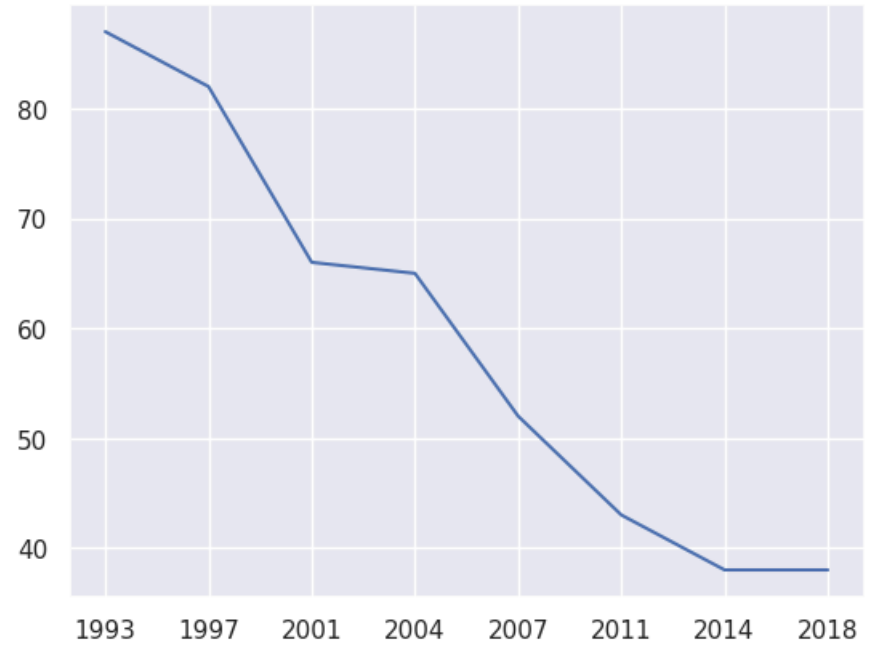
3.2 Lineplot

Drawing a simple line plot of Bangladesh infant mortality rate data

```
year = ["1993", "1997", "2001", "2004", "2007", "2011", "2014", "2018"]
im_rate = [87, 82, 66, 65, 52, 43, 38, 38]
```

A basic line plot only needs an x-axis list and a y-axis list passed to `plt.plot()`.

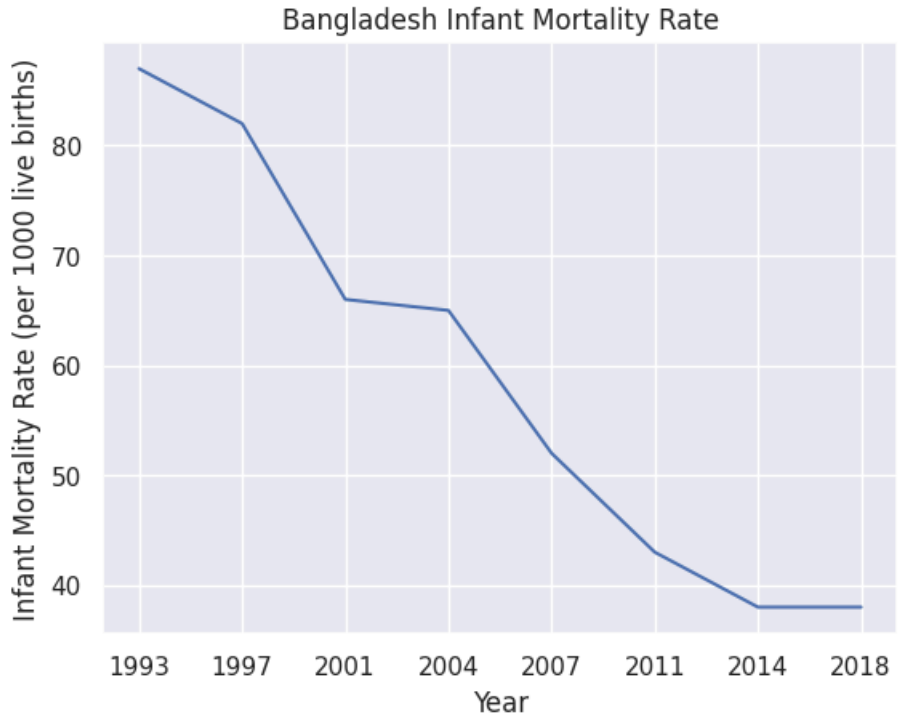
```
plt.plot(year, im_rate)
plt.show()
```



Adding title and label to the plot

```
plt.plot(year, im_rate)
plt.title("Bangladesh Infant Mortality Rate")
plt.xlabel("Year")
plt.ylabel("Infant Mortality Rate (per 1000 live births)")

plt.show()
```



matplotlib allows multiple styles. To use them, you first need to import the style class from matplotlib

```
from matplotlib import style
```

Run this command to get a list of all possible styles.

```
print(style.available)
```

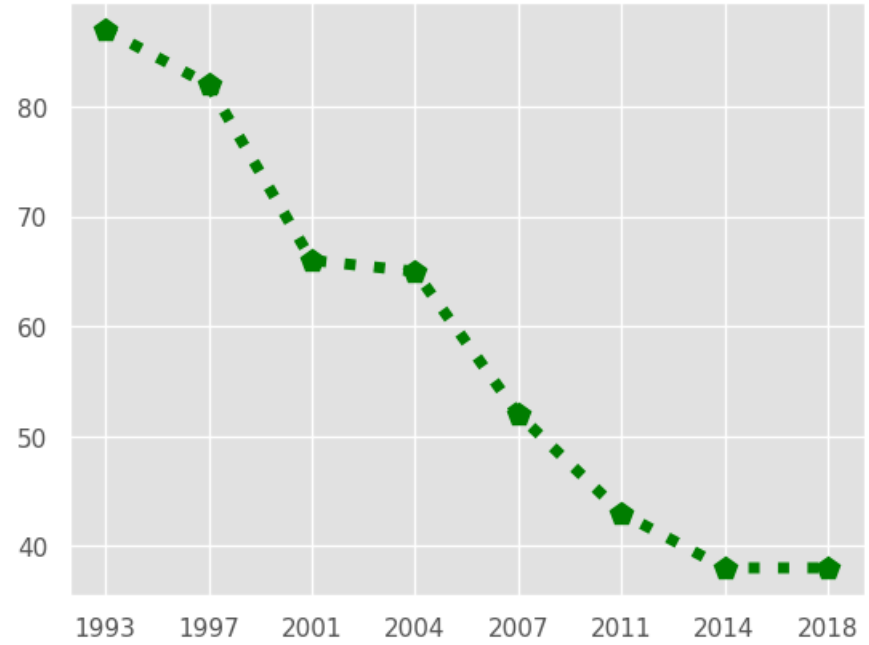
```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot',
```

Applying the `'ggplot'` style changes the overall look (background, grid, colors) of every plot drawn after this line.

```
# Use style.use(stylename) to set a style
style.use('ggplot')
```

plt.plot() allows few other options. Let's explore them. Try one of these as linestyle {'-', '--', '-.', ':', ''}. Try these as markers ('o', 'v', '^', '<', '>', '8', 's', 'p', '*', 'h', 'H', 'D', 'd', 'P', 'X').

```
plt.plot(year, im_rate, marker = 'p', color='green', linestyle=':', linewidth=5, markersize=10)
plt.show()
```



Let's compare this with another data. Under 5 mortality rate

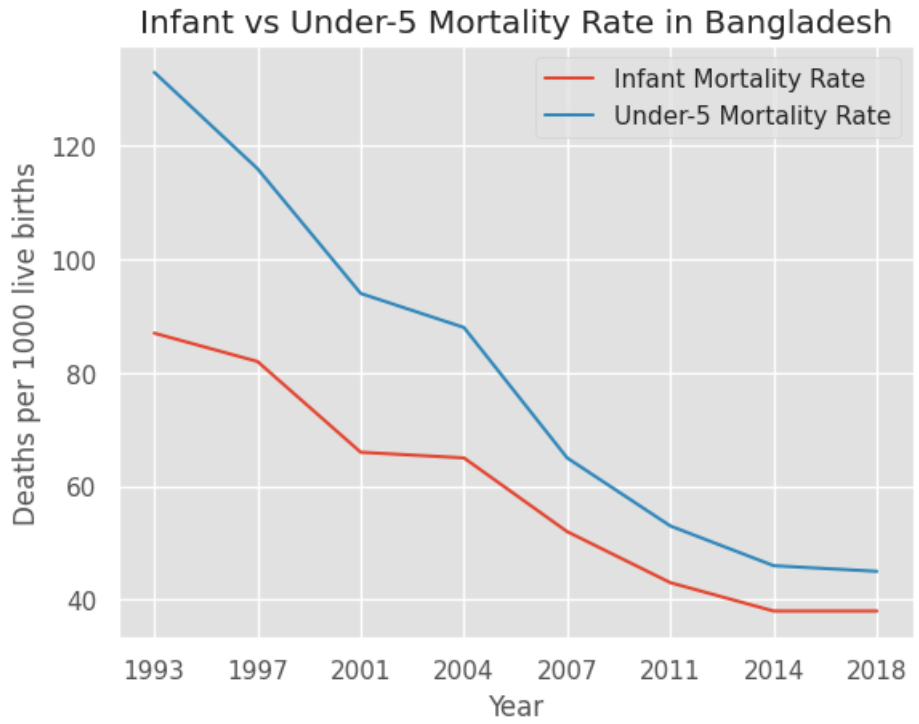
```
u5m_rate = [133, 116, 94, 88, 65, 53, 46, 45]
```

Plotting both series on the same axes lets us compare them. A legend is needed so we know which line is which.

```
plt.plot(year, im_rate)
plt.plot(year, u5m_rate)

plt.title("Infant vs Under-5 Mortality Rate in Bangladesh")
plt.xlabel("Year")
plt.ylabel("Deaths per 1000 live births")
plt.legend(["Infant Mortality Rate", "Under-5 Mortality Rate"])

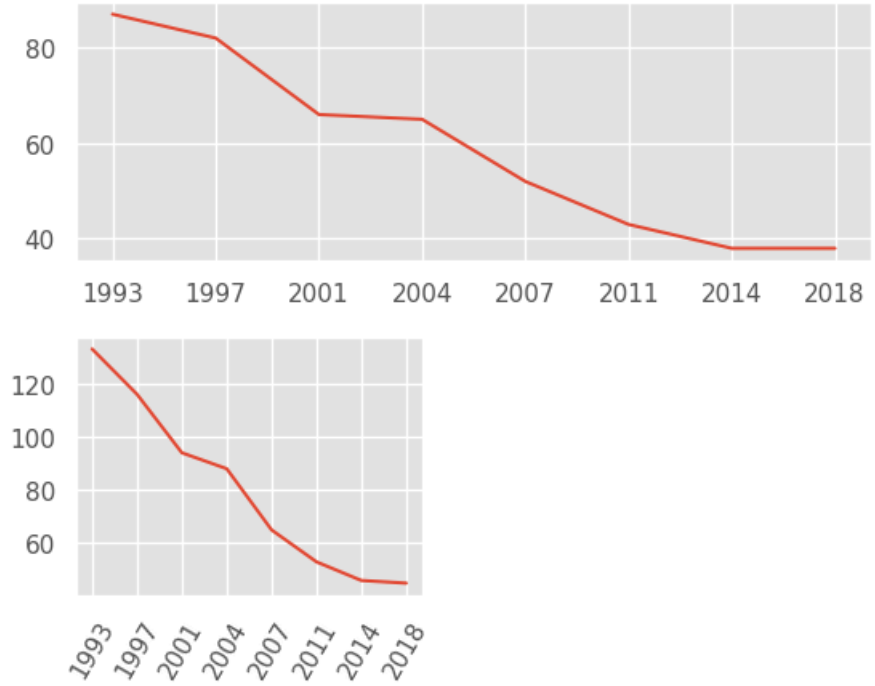
plt.show()
```



3.3 Subplot

Subplots allow us to show multiple plots combined in one plot

```
plt.subplots_adjust(wspace=0.3, hspace=0.3)
plt.subplot(2, 1, 1)
plt.plot(year, im_rate)
plt.subplot(2, 2, 3)
plt.plot(year, u5m_rate)
plt.xticks(rotation=60)
plt.show()
```



Now let's build a grid with one large plot on top spanning the full width, and two smaller plots side by side underneath.

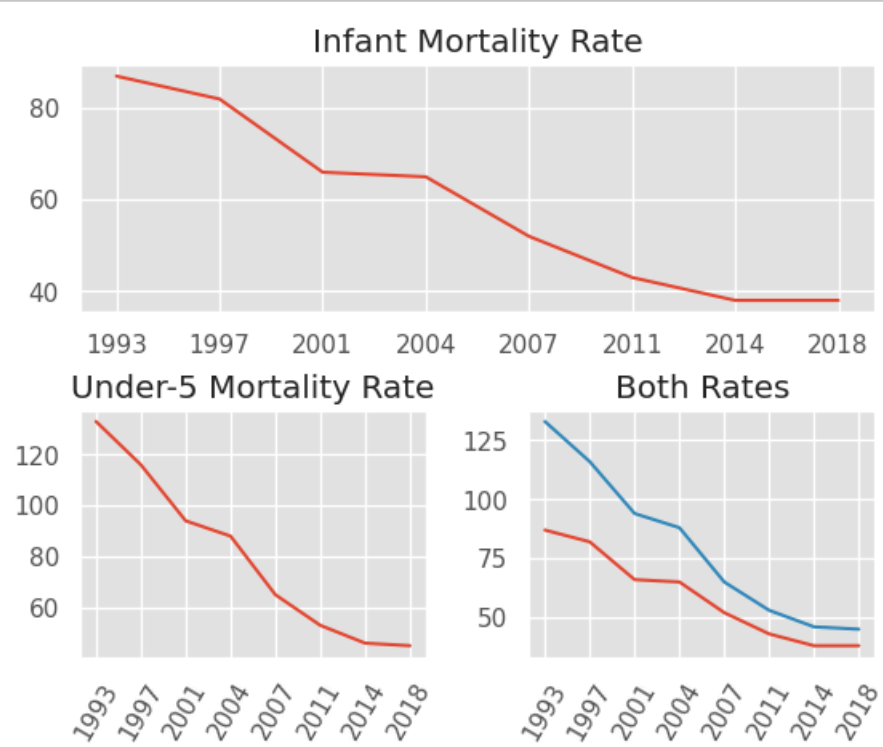
```
# Create a new subplot with 1 large row and 2 small columns
plt.subplots_adjust(wspace=0.3, hspace=0.4)

# Large plot spanning the full top row
plt.subplot(2, 1, 1)
plt.plot(year, im_rate)
plt.title("Infant Mortality Rate")

# Bottom-left small plot
plt.subplot(2, 2, 3)
plt.plot(year, u5m_rate)
plt.title("Under-5 Mortality Rate")
plt.xticks(rotation=60)

# Bottom-right small plot
plt.subplot(2, 2, 4)
plt.plot(year, im_rate)
plt.plot(year, u5m_rate)
plt.title("Both Rates")
plt.xticks(rotation=60)

plt.show()
```



3.4 Bar Plot

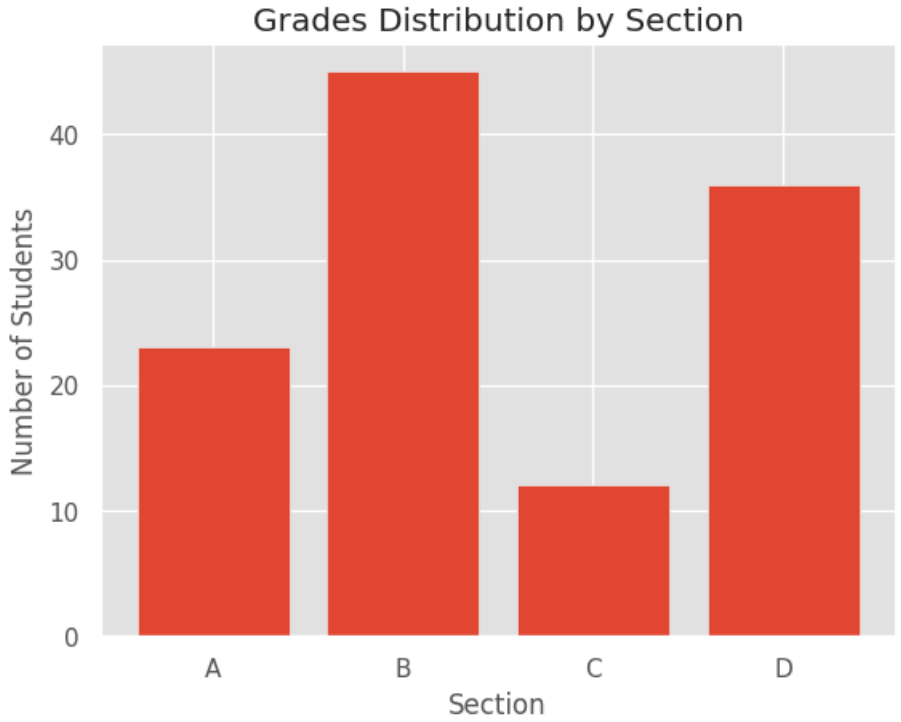
Bar plots are useful for comparing values across discrete categories.

```
# Data for the bar plot
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 12, 36]

# Creating the bar plot
plt.bar(categories, values)

# Add a title and labels
plt.title('Grades Distribution by Section')
plt.xlabel('Section')
plt.ylabel('Number of Students')

# Show the plot
plt.show()
```



3.5 Histogram

A histogram shows how a set of numeric values is distributed across bins. Here we look at the distribution of student marks.

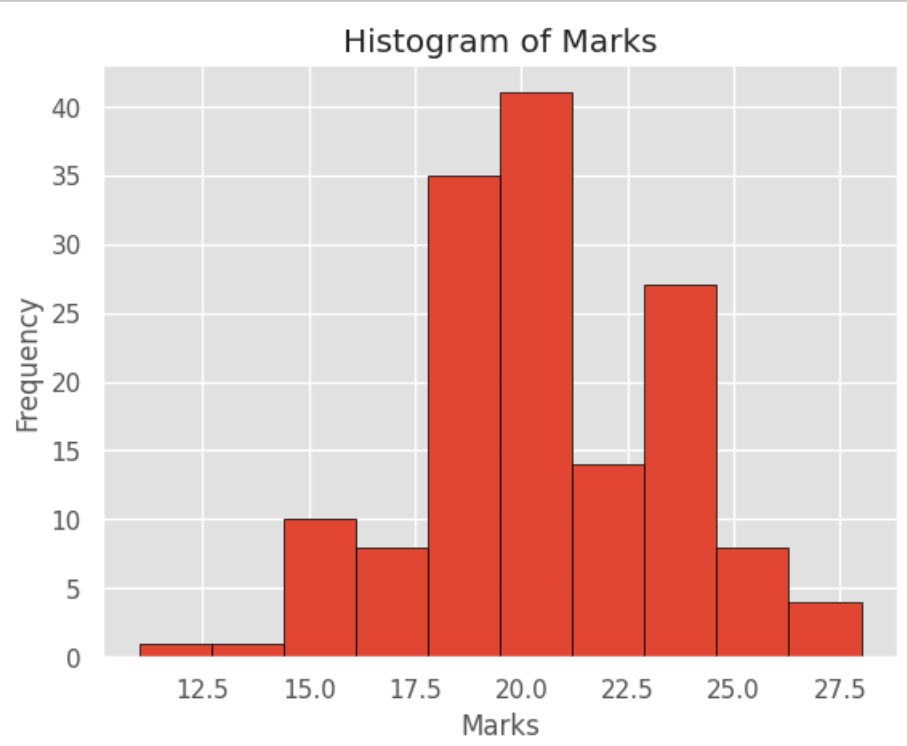
```
import matplotlib.pyplot as plt

# List of marks
marks = [24, 23, 24, 21, 21, 22, 19, 26, 18, 18, 13, 18, 23, 20, 22, 18, 26, 23, 24, 17, 17, 20, 26, 18, 21, 20, 19, 24, 22, 15, 20, 22, 16, 19, 22, 20, 2

# Create the histogram
plt.hist(marks, bins=10, edgecolor='black')

# Add a title and labels
plt.title('Histogram of Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')

# Show the plot
plt.show()
```



3.6 Contours

Contour plots are useful for visualizing a function of two variables as a 2D map of constant-value curves.

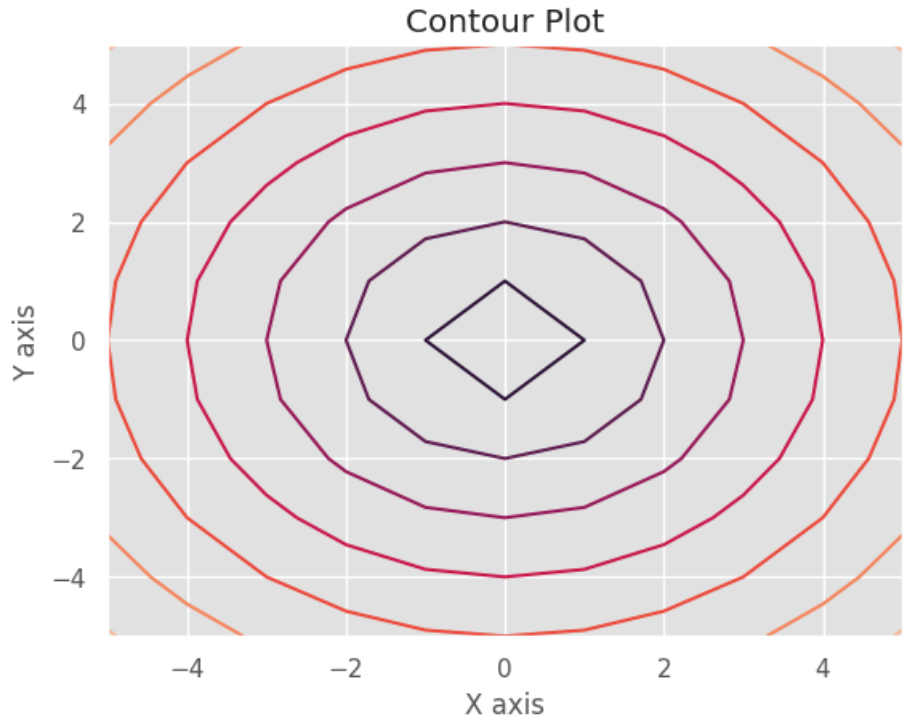
```
# Manually create the grid data
x = [i for i in range(-5, 6)]
y = [i for i in range(-5, 6)]

# Manually create Z values as a list of lists
Z = []
for i in x:
    row = []
    for j in y:
        z = (i**2 + j**2)**0.5  # Example function
        row.append(z)
    Z.append(row)

# Create the contour plot
plt.contour(x, y, Z)

# Add a title and labels
plt.title('Contour Plot')
plt.xlabel('X axis')
plt.ylabel('Y axis')
```

```
# Show the plot
plt.show()
```



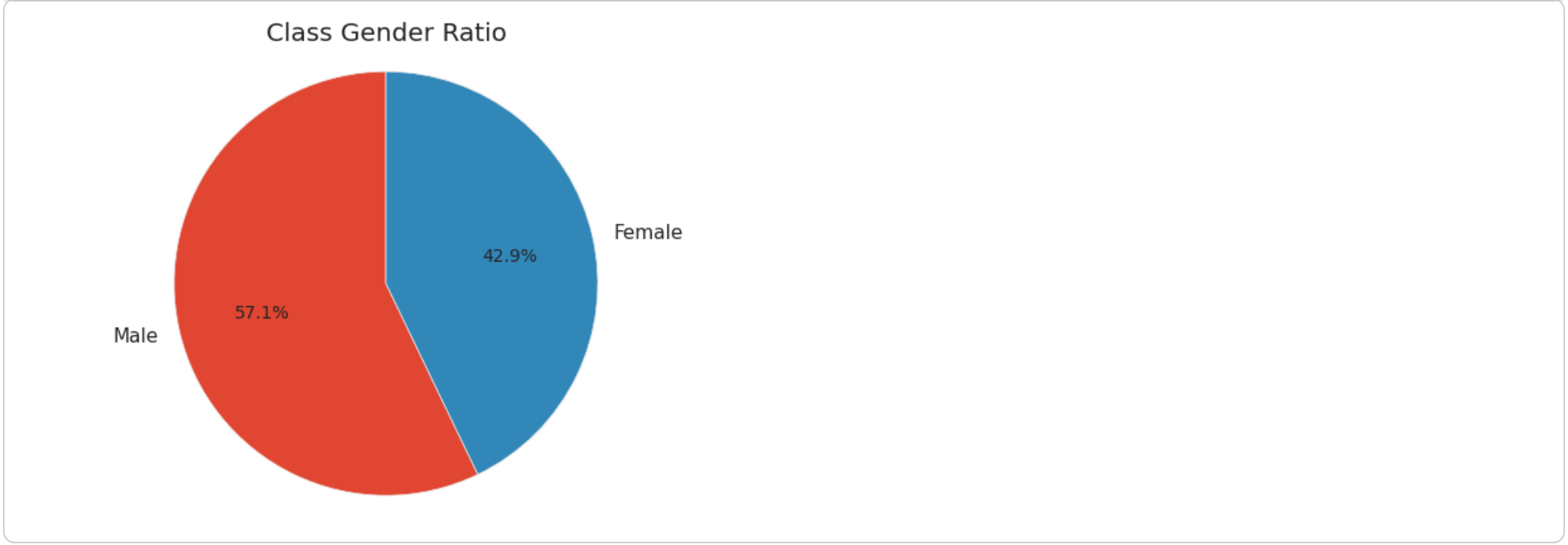
3.7 Pie Charts

Pie charts show each category's share of a whole. Here we visualize the male/female ratio of the class.

```
# Create a pie chart of your class' male female ratio
n_male = 20
n_female = 15

plt.pie([n_male, n_female], labels=['Male', 'Female'], autopct='%1.1f%%', startangle=90)
plt.title('Class Gender Ratio')
plt.axis('equal')

plt.show()
```



3.8 Getting Started with Seaborn

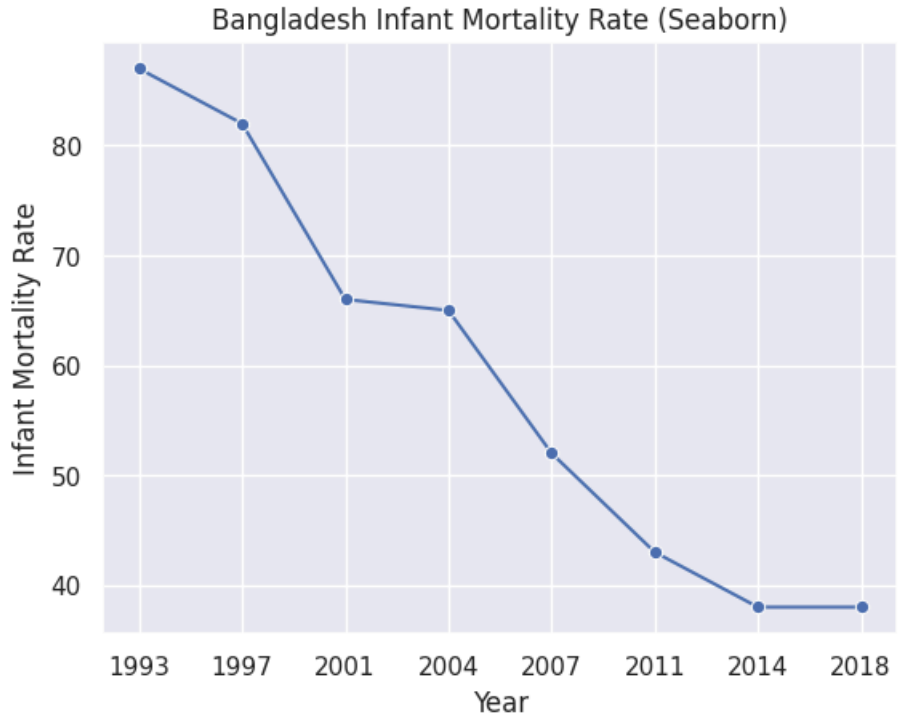
Seaborn plots directly accept lists or pandas objects, and add nicer default styling on top of matplotlib. Setting a theme once affects every seaborn plot that follows.

```
sns.set_theme(style='darkgrid')
```

3.8.1 Seaborn Lineplot

Redrawing the infant mortality line plot with seaborn.

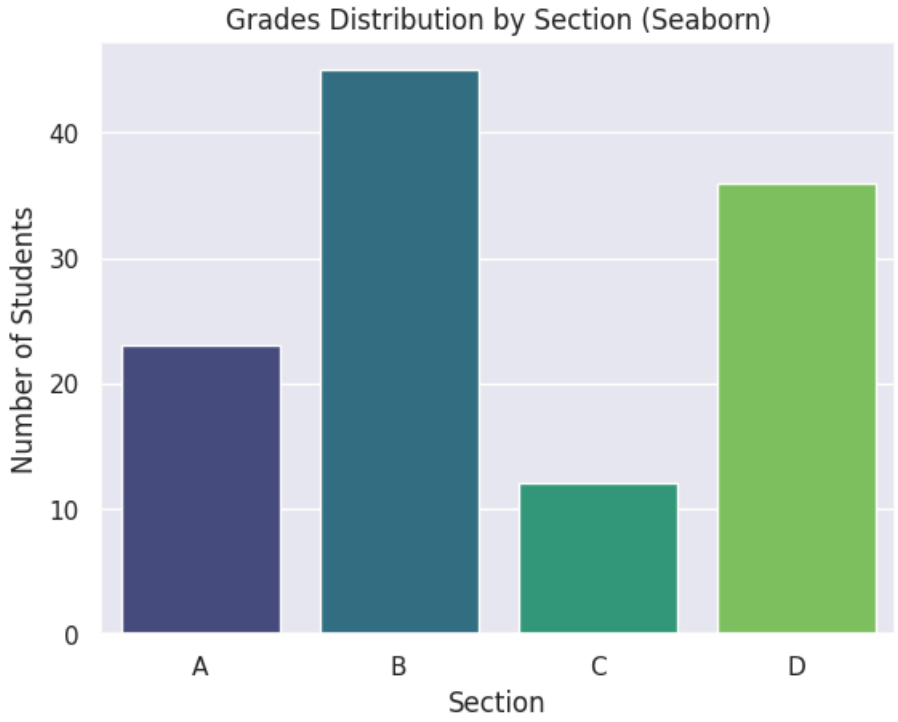
```
sns.lineplot(x=year, y=im_rate, marker='o')
plt.title('Bangladesh Infant Mortality Rate (Seaborn)')
plt.xlabel('Year')
plt.ylabel('Infant Mortality Rate')
plt.show()
```



3.8.2 Seaborn Bar Plot

The same bar chart data, drawn with `sns.barplot`.

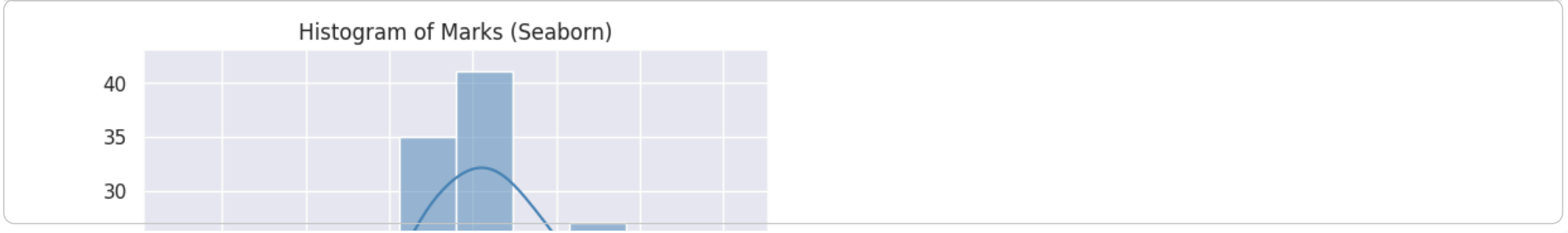
```
    sns.barplot(x=categories, y=values, hue=categories, palette='viridis', legend=False)
    plt.title('Grades Distribution by Section (Seaborn)')
    plt.xlabel('Section')
    plt.ylabel('Number of Students')
    plt.show()
```



3.8.3 Seaborn Histogram

`sns.histplot` draws a histogram and can optionally overlay a smooth density curve.

```
sns.histplot(marks, bins=10, kde=True, color='steelblue')
plt.title('Histogram of Marks (Seaborn)')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()
```


Histogram of Marks (Seaborn)
40
35
30